

A3

An introduction to reinforcement learning from human preferences

The goal of this assignment is to give you some hands-on experience with reinforcement learning from human preferences (RLHF). You will implement, compare, and contrast several different approaches for a robotics task in MuJoCo.

1 Reward engineering (6 pts writeup + 7 pts coding)

In Assignment 2 you applied PPO to solve an environment with a provided reward function. The process of deriving a reward function for a specific task is called reward engineering.

1.1 Written Questions (6 pts)

- a. **(2 pts) (written)** Why is reward engineering usually hard? What are potential risks that come with specifying an incorrect reward function? Provide an example of a problem and a reward function that appears to be adequate but may have unintended consequences.
 - b. **(2 pts) (written)** Read the description of the [Hopper environment](#). Using your own words, describe the goal of the environment, and how each term of the reward functions contributes to encourage the agent to achieve it. (Do you agree with this reward function? Would you change it? How?)
 - c. **(2 pts) (written)** By default, the episode terminates when the agent leaves the set of "healthy" states. What do these "healthy" states mean? Name one advantage and one disadvantage of this early termination.
- a. Reward engineering is difficult because, reward functions are approximations of our true goals, and translating complex goals into a precise numerical reward signal is non-trivial and error-prone. Mistakes in specification can lead to unintended behaviors, of which a common one is "reward hacking," where the agent exploits loopholes in the reward to maximize it without achieving human engineers real goal.
- For example, if a cleaning robot's reward is based on the number of dirt spots cleaned, it might scatter dirt to gain more reward instead of cleaning effectively. We have also seen an example in A1 about a traffic optimization agent.
- b. The goal is for the agent to control a 2D one-legged robot to "hop" (because it is only a leg) forward as far and as fast as possible without falling.
- The reward function has three terms:
- a forward velocity reward encouraging the agent to move forward (right).
 - a healthy state bonus rewarding balance and preventing falls.
 - a control cost penalizing large or abrupt actions to promote smooth and efficient movement.

This combination encourages fast, stable hopping with realistic joint control.

I think I agree with this reward function, it accounts for most parts of the real goal, and it seems robust enough. I would maybe add another penalty that says the energy usage (battery etc.) cannot be too large, by using a $\Sigma |\text{torque} * \text{angular_velocity}|$. It may complicate learning or turns out to be not actually effective though.

c. "Healthy" states are those where the robot's joint angles, velocities, and torso position remain within bounds. These are possibly arbitrarily set ranges to mimic human bodily balances, and thus helping maintain upright and balanced local motions.

One advantage I can think of is:

- Such early termination encourages the agent to maintain stability and avoid catastrophic failure. It reduces waste of computation by resetting poor trajectories early.

One disadvantage might be:

- It can be overly conservative by cutting episodes short when recovery might still be possible, potentially limiting learning of recovery behaviors. Or it prevents a policy where an agent "fall" in a weird way that's not actually catastrophic for the task of moving forward.

1.2 Coding Questions (7 pts)

d. (2 pts) (**coding**) Use the provided starter code to train a policy using PPO to solve the Hopper environment for 3 different seeds. Do this with and without early termination.

```
python ppo_hopper.py [--early-termination] --seed SEED
```

e. (2 pts) (**written**) Attach here the plot of the episodic returns along training, with and without early termination. You can generate the plot by running

```
python plot.py --directory results --seeds SEEDS
```

where **SEEDS** is a comma-separated list of the seeds you used. Comment on the performance in terms of training epochs and wall time. Is the standard error in the average returns high or low? How could you obtain a better estimate of the average return on Hopper achieved by a policy optimized with PPO?

f. (2 pts) (**written**) Pick one of the trained policies and render a video of an evaluation rollout.

```
python render.py --checkpoint [PATH TO MODEL CHECKPOINT]
```

Does the agent successfully complete the assigned task? Does it complete it in the way you would expect it to, or are you surprised by the agent behavior?

g. (1 pts) (**written**) Render another video for another policy. How do the two rollouts compare? Do you prefer one over the other?

d. Completed. Used seed numbers 11, 22, 33. Commands were:

```
python ppo_hopper.py --seed 11
```

```
python ppo_hopper.py --early-termination --seed 11
```

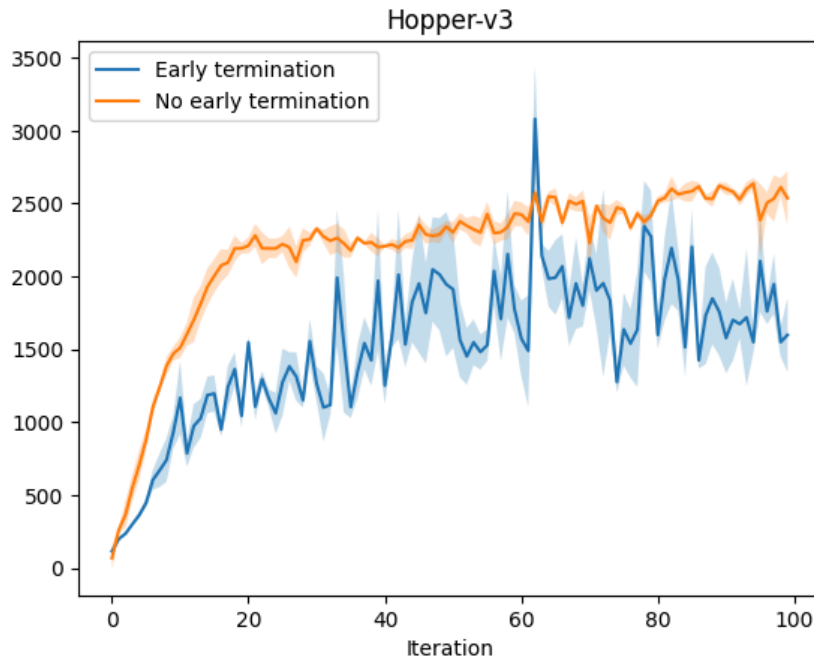
```
python ppo_hopper.py --seed 22
```

```
python ppo_hopper.py --early-termination --seed 22
```

```
python ppo_hopper.py --seed 33
```

```
python ppo_hopper.py --early-termination --seed 33
```

e. Below is a plot of episodic returns along training, with and without early termination.



Generated by running

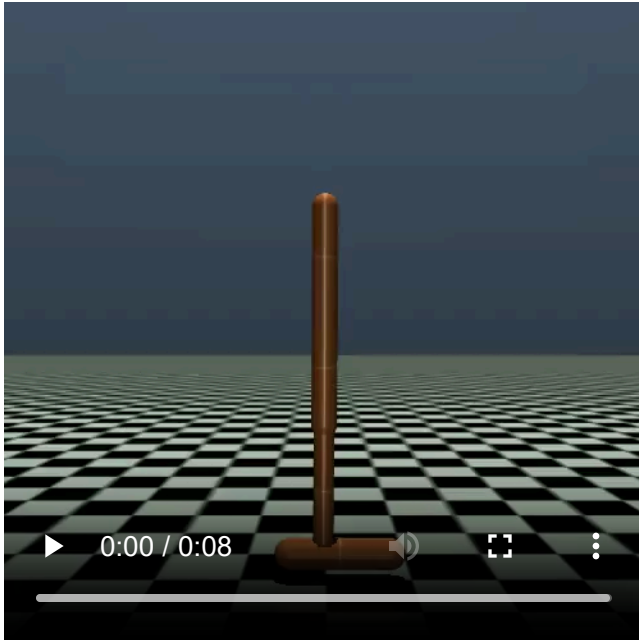
```
python plot.py --directory results -o output --seeds 11,22,33.
```

Observations in training epochs:

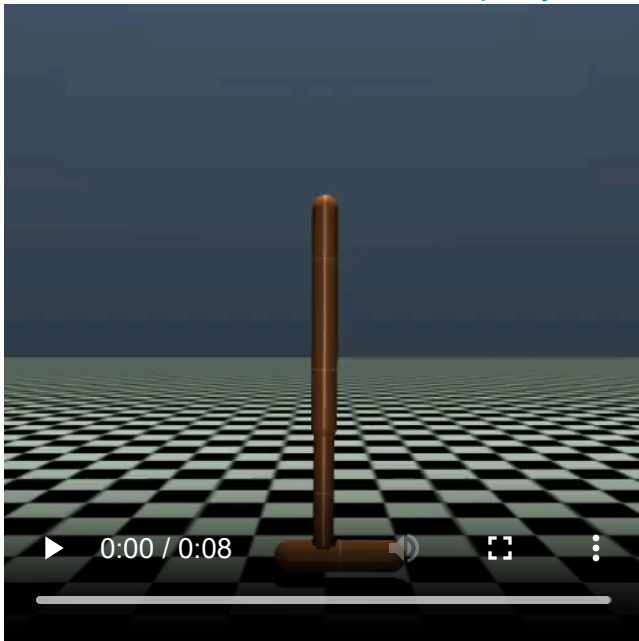
- The agent learns (gain returns) faster at early stages when there is no early termination. The curve plateaus sooner.
- The agent is more conservative (gain less returns after learning plateaus) when there is early termination.
- Returns fluctuate more at later stages in the with-early-termination case.

The wall time of trainings with no early termination was about 770s on my machine, and that of trainings with early termination was about 650s. So the agent seems to learn less from the environment when there is early termination. The standard error in average returns is relatively high, due to using only 3 random seeds. To improve return estimates, I think we could train using at least 5–10 seeds and evaluate each checkpoint with multiple rollout episodes to better capture performance variance across runs.

f. This is rendered video of the trained policy with seed 11, no early termination.



This is rendered video of the trained policy with seed 11, with early termination.



Generated by running

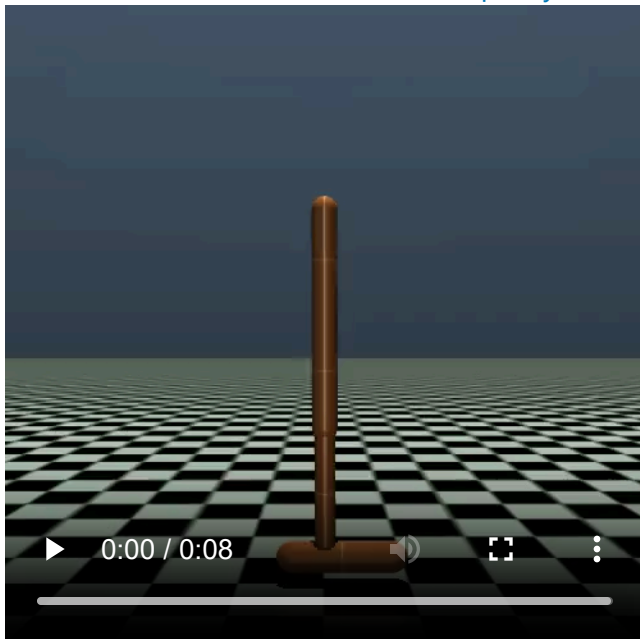
```
python render.py --checkpoint results/Hopper-v3-early-termination=False-seed=11/model.zip  
python render.py --checkpoint results/Hopper-v3-early-termination=True-seed=11/model.zip.
```

In both of these, I would say the agent successfully completes the assigned task, because they have indeed hopped forward, although the with-early-termination agent was cut short because of the unhealthy posture. The agent in the no early termination case is not completing the task in the way I expected, because its local motions are unnatural from a human's perspective. I was surprised that it continues to move forward using that kind of postures, which looks anti-physics when I expected to see human-like gaits.

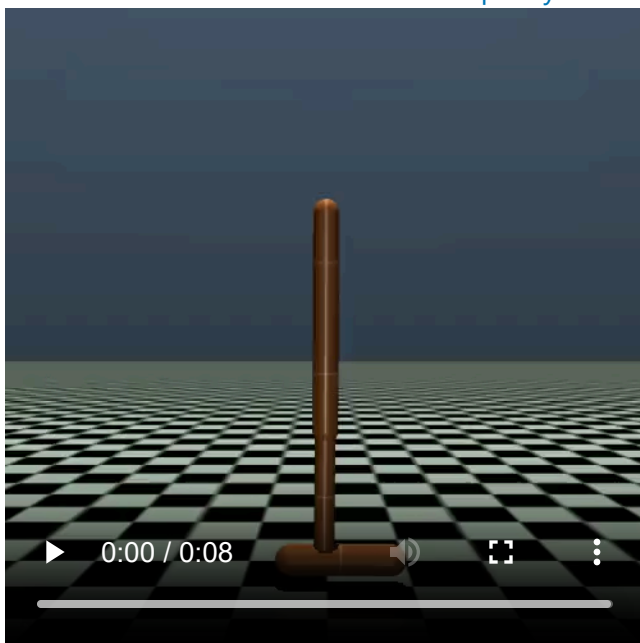
But I suppose it makes sense in the simplified Hopper environment. The Mujoco physics engine has quirks that agents can exploit (e.g., no fatigue, friction artifacts). Also, since there's no penalty for looking weird (unless it breaks healthy state rules), the agent is free to learn whatever works.

Another surprise is that it tends to "stand back up" at some point, which does not appear in the other two trained policies with no early termination.

g. This is rendered video of the trained policy with seed 22, no early termination.



This is rendered video of the trained policy with seed 22, with early termination.



Compared with the rollouts of policy with seed 11, the rollouts of policy with seed 22 are not very different. I like rollouts of policy with seed 22 a bit more, because from the video of rollout with early termination, the motions it produces looks a bit more stable near the end of the training: slightly less global vertical bouncing, and traveled a bit further with seemingly smoother control.

2 Learning from preferences (5 pts writeup + 25 pts coding)

In the previous part you trained multiple policies from scratch and compared them at the end of training. In this section, we will see how we can use human preferences on two roll-outs to learn a reward function.

We will follow the framework proposed by (Christiano et al. 2017). A reward function $r : \mathcal{O} \times \mathcal{A} \rightarrow \mathbb{R}$ defines a preference relation $\succ$ if for all trajectories $\sigma^i = (o_t^i, a_t^i)_{t=0, \dots, T}$ we have that

$$((o_0^1, a_0^1), \dots, (o_T^1, a_T^1)) \succ ((o_0^2, a_0^2), \dots, (o_T^2, a_T^2))$$

whenever

$$r(o_0^1, a_0^1) + \dots + r(o_T^1, a_T^1) > r(o_0^2, a_0^2) + \dots + r(o_T^2, a_T^2).$$

Following the Bradley-Terry preference model (Bradley and Terry 1952), we can calculate the probability of one trajectory σ^1 being preferred over σ^2 as follows:

$$\hat{P}[\sigma^1 \succ \sigma^2] = \frac{\exp \sum \hat{r}(o_t^1, a_t^1)}{\exp \sum \hat{r}(o_t^1, a_t^1) + \exp \sum \hat{r}(o_t^2, a_t^2)},$$

where $\hat{r}$ is an estimate of the reward for a state-action pair. This is similar to a classification problem, and we can fit a function approximator to $\hat{r}$ by minimizing the cross-entropy loss between the values predicted with the above formula and ground truth human preference labels $\mu(1)$ and $\mu(2)$:

$$\text{loss}(\hat{r}) = - \sum_{(\sigma^1, \sigma^2, \mu) \in \mathcal{D}} \mu(1) \log \hat{P}[\sigma^1 \succ \sigma^2] + \mu(2) \log \hat{P}[\sigma^2 \succ \sigma^1].$$

Once we have learned the reward function^{*}, we can apply any policy optimization algorithm (such as PPO) to maximize the returns of a model under it.

Note

* Recent work on RLHF for reinforcement learning suggests that the pairwise feedback provided by humans on partial trajectories may be more consistent with regret, and that the learned reward function may be better viewed as an advantage function. See Knox et al. AAAI 2024 "Learning optimal advantage from preferences and mistaking it for reward." <https://openreview.net/forum?id=euZXhbTmQ7>

2.1 Written questions (5 pts)

a. (5 pt) Let $\hat{r}(o, a) = \phi_w(o, a)$. Calculate $\nabla_w \text{loss}(\hat{r}(o, a))$.

a. For convenience, denote:

$$R_1 = \sum_t \phi_w(o_t^1, a_t^1), \quad R_2 = \sum_t \phi_w(o_t^2, a_t^2).$$

So:

$$\hat{P}[\sigma^1 > \sigma^2] = \frac{e^{R_1}}{e^{R_1} + e^{R_2}}, \quad \hat{P}[\sigma^2 > \sigma^1] = \frac{e^{R_2}}{e^{R_1} + e^{R_2}}.$$

The gradient of a sum is the sum of gradients. So we can look at the loss for a single pair $(\sigma^1, \sigma^2, \mu)$:

$$\ell(w) = -\mu(1) \log \left(\frac{e^{R_1}}{e^{R_1} + e^{R_2}} \right) - \mu(2) \log \left(\frac{e^{R_2}}{e^{R_1} + e^{R_2}} \right).$$

Apply log rules:

$$\ell(w) = -\mu(1)(R_1 - \log(e^{R_1} + e^{R_2})) - \mu(2)(R_2 - \log(e^{R_1} + e^{R_2})).$$

Group terms:

$$\ell(w) = -\mu(1)R_1 - \mu(2)R_2 + (\mu(1) + \mu(2)) \log(e^{R_1} + e^{R_2}).$$

Now differentiate w.r.t. w :

$$\nabla_w \ell(w) = -\mu(1)\nabla_w R_1 - \mu(2)\nabla_w R_2 + (\mu(1) + \mu(2))\nabla_w \log(e^{R_1} + e^{R_2}). \quad (1)$$

Recall:

$$\nabla_w \log(e^{R_1} + e^{R_2}) = \frac{e^{R_1}\nabla_w R_1 + e^{R_2}\nabla_w R_2}{e^{R_1} + e^{R_2}} = \hat{P}[\sigma^1 > \sigma^2]\nabla_w R_1 + \hat{P}[\sigma^2 > \sigma^1]\nabla_w R_2$$

, and plug it into (1):

$$\nabla_w \ell(w) = -\mu(1)\nabla_w R_1 - \mu(2)\nabla_w R_2 + (\mu(1) + \mu(2)) \left(\hat{P}[\sigma^1 > \sigma^2]\nabla_w R_1 + \hat{P}[\sigma^2 > \sigma^1]\nabla_w R_2 \right).$$

Factor terms:

$$\nabla_w \ell(w) = \left(-\mu(1) + (\mu(1) + \mu(2))\hat{P}[\sigma^1 > \sigma^2] \right) \nabla_w R_1 + \left(-\mu(2) + (\mu(1) + \mu(2))\hat{P}[\sigma^2 > \sigma^1] \right) \nabla_w R_2.$$

By using the fact that

$$\mu(1) + \mu(2) = 1$$

, it can be further simplified as:

$$\nabla_w \text{loss}(\hat{r}) = \sum_{(\sigma^1, \sigma^2, \mu) \in D} \left((\hat{P}[\sigma^1 > \sigma^2] - \mu(1))\nabla_w R_1 + (\hat{P}[\sigma^2 > \sigma^1] - \mu(2))\nabla_w R_2 \right).$$

2.2 Coding questions (25 pts)

In this problem we are trying to solve the same task as in the previous part, but this time we will learn a reward function from a dataset of preferences rather than manually specifying a reward function.

- b. (5 pt) Load one of the samples from the preference dataset we provide you, and render a video of the two trajectories using the following command

```
python render.py --dataset [PATH TO PREFERENCE DATASET FILE] --idx IDX
```

where `IDX` is an index into the preference dataset (if omitted a sequence will be chosen at random). Bear in mind that each sequence in the dataset has 25 timesteps, which means that the resulting videos will have 0.2 seconds. Take note of which sequence was labeled as preferred (this information will appear in the name of the generated videos, but for the coming parts it is helpful to know that 0 means the first sequence was preferred, 1 means the second one, and 0.5 means neither is preferred over the other). Do you agree with the label (that is, if shown the two trajectories, would you have ranked them the same way they appear in the dataset, knowing that we are trying to solve the Hopper environment)?

- c. (3 pt) Repeat the previous question for 4 more samples, keeping track of whether you personally agree with the dataset preference label. Use this to estimate how much you agree with whoever ranked the trajectories. How much did you get? Based on this agreement estimate, would you trust a reward function learned on this data?
- d. (8 pt) Implement the functions in the `RewardModel` class (`run_rlhf.py`), which is responsible for learning a reward function from preference data.

e. (5 pt) Train a model using PPO and the learned reward function with 3 different random seeds. Plot the average returns for both the original reward function and the learned reward function and include it in your response.

```
python plot.py --rlhf-directory results_rlhf --output \
results_rlhf/hopper_rlhf.png
```

Do the two correlate?

f. (1 pt) Is the learned reward function identifiable?

g. (3 pt) Pick one of the policies and render a video of the agent behavior at the end of training.

```
python render.py --checkpoint [PATH TO MODEL CHECKPOINT]
```

How does it compare to the behavior of the agent generated by policies trained from scratch? How does it compare to the demonstrations you've seen from the dataset?

b. After inspecting that the index range is 0 to 9999, I used this command

```
python render.py --dataset data/prefs-hopper.npz --idx 456
```

to render the sample with index 456. For this sample, the result is equally preferred. I watched the videos, and indeed could not tell the difference between the two rollouts. So I would say I agree with the labeling.

c. I rendered 4 more samples using the same command without specifying indices. The randomly chosen indices were 394, 6866, 7560 and 6851. Here is a table recording the difference between my labeling and the person who labeled these samples.

index	embeded preference	my preference	do we agree?
456	0.5 equally preferred	equally preferred	yes
394	0.5 equally preferred	equally preferred	yes
6866	0.0 1st preferred	equally preferred	no
7560	0.0 1st preferred	1st preferred	yes
6851	0.0 1st preferred	1st preferred	yes

Based on this agreement estimate, I think I could reasonably trust a reward function learned on this data.

d. All of `__init__()`, `forward()`, `compute_reward()`, and `update()` have been implemented.

e. I used these commands to run the *RLHF* procedure

```
python run_rlhf.py --seed [SEED]
```

for three different seeds 11, 22 and 33, to be consistent with previous work. Other command line arguments like

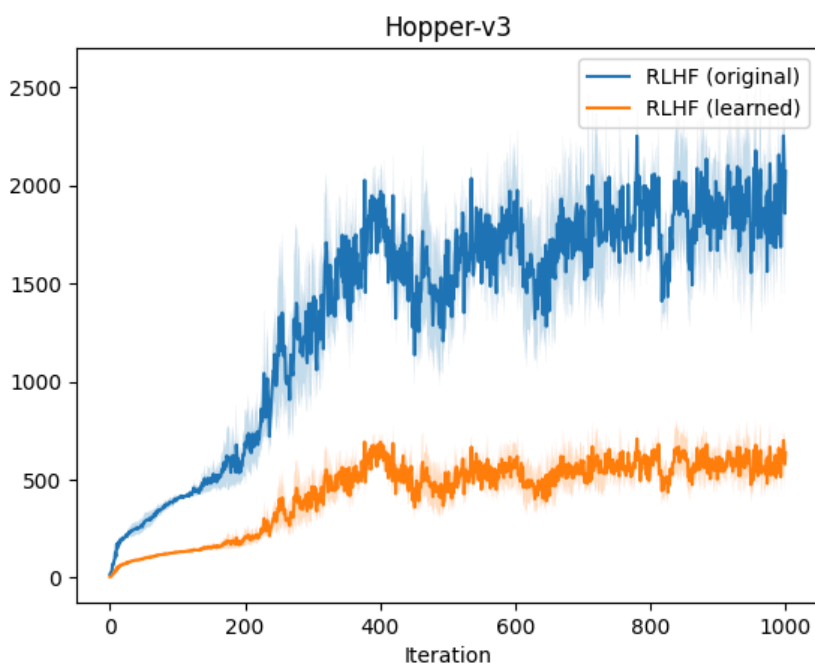
```
--env-name
--dataset-path
--reward-min
--reward-max
--reward-model-hidden-dim
--reward-model-steps
--reward-model-batch-size
--rl-steps
--eval-period
--num-eval-episodes
```

are not inputted because the defaults are good enough.

Using command

```
python plot.py --rlhf-directory results_rlhf --output results_rlhf/hopper_rlhf.png --seeds 11,22,33
```

, below is the plotted aggregated graph of the 6 curves put together.



Correlation

I observe pretty strong correlation. Across all three seeds, the average learned return and average original return:

- Increase at the same training steps.
- Plateau around the same time.
- Fluctuate in sync during the plateau phase.

It may be worth noting that the learned return appears like a “vertically compressed” version of the original return. This is likely the effect of the *RLHF* rewards being scaled.

This indicates that:

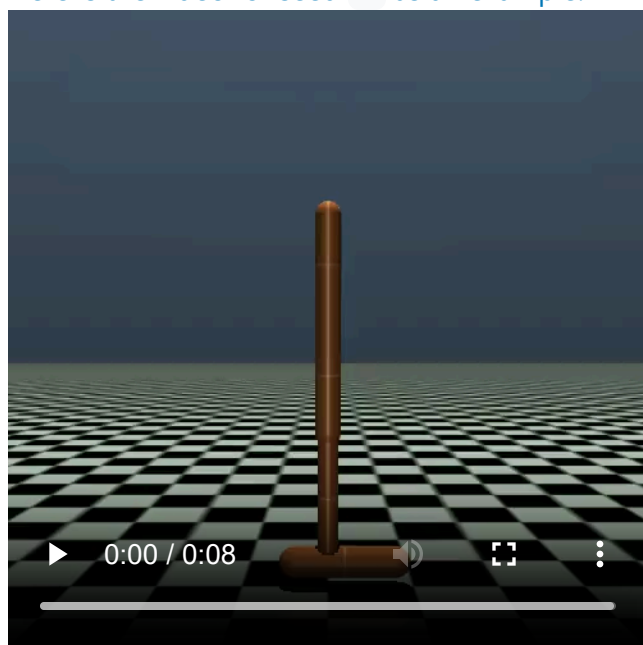
- The reward model captures the relative quality of behaviors well, since trends in learned return correlate with trends in true return.

- The reward model does not quite match the scale or exact values of the environment's reward function, which is expected, since it was trained only on pairwise preferences, with a scaled-down (remapped into range `[self.r_min, self.r_max]`) proxy of the real reward. But the trend preservation tells us it is a good proxy.
- f. No, the learned reward function is not identifiable because multiple reward functions, e.g. differing by positive scaling or translation, can explain the same human preferences.
- g. I actually rendered videos for all three seeds using commands:

```
python render.py --checkpoint results_rlhf/Hopper-v3-rlhf-seed=[SEED]/model.zip
```

and visually compared them with videos for policies trained from scratch using the true Hopper reward (done for problem 1.2), who used the same set of seeds.

Here is the video for seed 22 as an example.



Comparison

- Compared to PPO agents trained from scratch, the RLHF-trained agents exhibited more natural and stable behavior. Their legs remained upright (rather than dragging or crawling), and the agent covered greater distances with smoother hops.
- These traits were more aligned with the human demonstrations seen in the dataset, suggesting that the learned reward better captures human-preferred locomotion than the environment's native reward function does.

3 Direct preference optimization (25 pts coding)

In the previous question we saw how we could train a model based on preference data. However, suppose you are given a pre-trained model and the corresponding preference data. Ideally, you would like to optimize the model directly on the preference data, instead of having to learn a reward function, and then run PPO on it. That is the idea behind direct preference optimization (DPO) ([Rafailov et al. 2023](#)). The algorithm proposed in the original paper allows us to skip the reward learning and reinforcement learning steps, and optimize the model directly from preference data by optimizing the following loss:

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right],$$

where π_{ref} is the policy from which we sampled y_w and y_l given x , π_{θ} is the policy we are optimizing, and σ is the sigmoid function.

We will use DPO in this question as well. To provide some context, let us consider the general approach for RLHF for text generation:

1. Train a large language model (LLM) to do next token prediction given a context (the tokens that came previously).
2. Given a fixed context x , generate possible next token sequence predictions y_1 and y_2 , and store the triple (x, y_1, y_2) .
3. Ask human supervisors to rank y_1 and y_2 given x according to individual preference.
4. Update the LLM to maximize the probability of giving the preferred answers using reinforcement learning.

In a similar way, given an observation x we could have two ranked sequences of actions $a_{1:T}^1$ and $a_{1:T}^2$, train the model to generate the preferred sequence of actions, and then execute them all^{*}. If the length of the generated action sequence is equal to the environment time horizon, this is called open-loop control. However, this approach lacks robustness, since the plan of actions will not change in response to disturbances or compounding errors. Instead, we are going to adapt this into a more robust scheme by training our policy to predict a sequence of actions for the next T time steps (where T is the length of the trajectories in our preference dataset), but only take the first action in the plan generated by the policy. In this way, we re-plan our actions at every time step, ensuring the ability to respond to disturbances.

Note

^{*} To understand why we are considering sequences of actions rather than a single action for the next time, recall that 25 actions corresponded to 0.2 seconds of video. If you found it difficult to rank a sequence of 25 actions based on such a short video, imagine ranking the effect of a single action!

3.1 Coding questions (25 pts)

- a. (9 pts) Implement the `ActionSequenceModel` class instance methods. When called, the model should return a probability distribution for the actions over the number of next time steps specified at initialization. Use a multivariate normal distribution for each action, with mean and standard deviation predicted by a neural network (see the starter code for more details)^{*}.

Note

^{*} We have prepared a [notebook](#) to illustrate the behavior of `torch.distributions.Independent`.

- b. (3 pts) Implement the `update` method of the `SFT` class. This class will be used to pre-train a policy on the preference data by maximizing the log probabilities of the preferred actions given the observations in the dataset.
- c. (5 pts) Implement the `update` method of the `DPO` class. This should minimize the DPO loss described above.
- d. (5 pts) Run DPO for 3 different random seeds (you may want to tweak the number of DPO steps to get better results), and plot the evolution of returns over time.

```
python plot.py --dpo-directory results_dpo --output \
results_dpo/hopper_dpo.png
```

Include that plot in your response. How does it compare to the returns achieved using RLHF? Comment on the pros and cons of each method applied to this specific example.

- e. (3 pts) Render a video of an episode generated by the pre-trained policy, and a video of an episode generated by the policy tuned by DPO.

```
python render.py --dpo --checkpoint [PATH TO MODEL CHECKPOINT]
```

How do they compare?

- a. All of `__init__()`, `forward()`, `distribution()`, and `act()` have been implemented.
- b. Implemented.
- c. Implemented.
- d. Initially used these parameters:

```
python run_dpo.py --seed 11 --num-dpo-steps 50000 --dpo-lr 3e-7 --eval-period 1000
python run_dpo.py --seed 22 --num-dpo-steps 50000 --dpo-lr 3e-7 --eval-period 1000
python run_dpo.py --seed 33 --num-dpo-steps 50000 --dpo-lr 3e-7 --eval-period 1000
```

, but the results of trainings with seeds 22 and 33 showed decreasing return after some time steps. So I did many tweaks such as:

- Different `--num-dpo-steps` values to find the roughly “optimal” number just before the returns started to decline.
- Different `--dpo-lr` values for the chosen seeds, and find a best one for each seed.

Here are the parameters that were settled on:

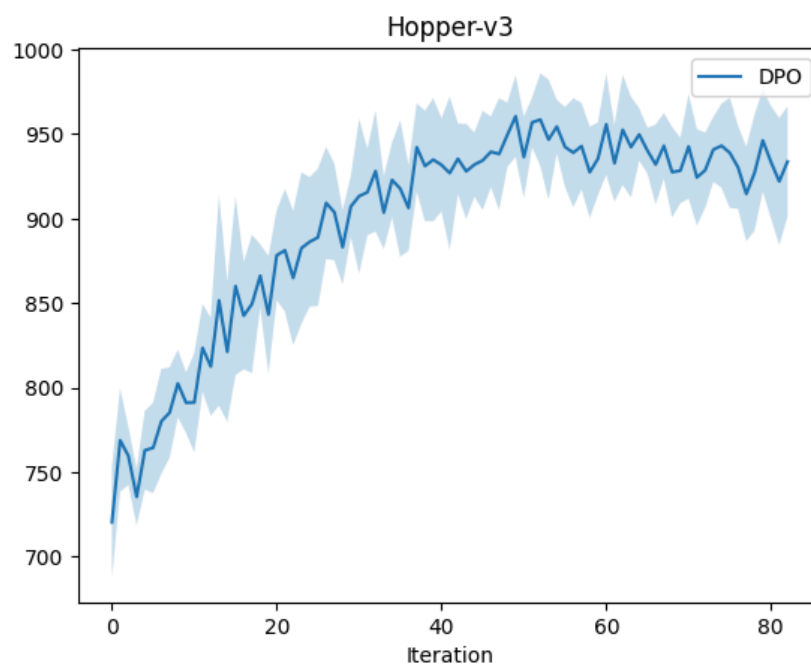
```
python run_dpo.py --seed 11 --num-dpo-steps 83000 --dpo-lr 3e-7 --eval-period 1000
python run_dpo.py --seed 22 --num-dpo-steps 83000 --dpo-lr 2e-7 --eval-period 1000
python run_dpo.py --seed 33 --num-dpo-steps 83000 --dpo-lr 3e-7 --eval-period 1000
```

. The idea was to fix a good `num-dpo-steps` as much as possible, and then find a good `dpo-lr` that brings the best out of each subset of data under this `num-dpo-steps` value.

Using the provided script

```
python plot.py --dpo-directory results_dpo --output results_dpo/hopper_dpo.png --seeds 11,22,33
```

, the following aggregated plot was made.



Observations

- Speed: DPO gets up to good behavior far faster than RLHF.
- Final performance: DPO achieves higher mean returns on Hopper under the same horizon and evaluation protocol.
- Hyperparameter sensitivity: DPO required manual tuning (learning rate and number of training steps) to avoid collapse after the plateau.

Pros-cons comparison

DPO pros

- Learns end-to-end directly from preference data; skips the two-stage reward model + PPO loop.
- Rapid improvement in early training.
- Higher final returns on this Hopper task (≈ 950 vs. 600), but may be due to reward-scale differences.

DPO cons

- Sensitive to hyperparameters (learning rate, number of steps).
- No explicit reward model means less interpretability of learned "reward."
- May overfit preferences if not carefully regularized.

RLHF pros

- Modular: explicit reward model that you can inspect, visualize, or reuse.
- PPO adds robustness via environment interaction and on-policy exploration.
- More stable across seeds with default settings.

RLHF cons

- Slower: two phases, reward fitting + PPO, each require many steps.
- Lower returns in this Hopper problem (again may partly incorporating differences in reward scaling).

e. Considering that both policies in each pair are stochastic, I rendered several for each. For example, under the folder with seed 11, 3 videos with model_sft.pt and 3 videos with model.pt were rendered and compared. Then the human mind tried to qualitatively catch differences in the trends.

DPO's improving effect may not be obvious at all in some of these. I observed that some datasets tend to result in SFT policies that produces worse rollouts, and some tend to result in better ones, so I suppose varied effects of DPO is not abnormal. Also the numerical rewards improvement was only about 350, which is indeed not that much.

But in general, there is a trend of DPO fine-tuned policy rollouts exhibiting slightly more balanced gaits and slightly longer hopping distances.

Additionally, when compared with rollout videos with the RLHF policies in previous problems, RLHF seems to train better policies than DPO. This could be because the β we use for DPO is pretty small here, and also as noted above, data quality affects DPO results. Besides, visual observations do not always reflect the whole situation: numerical rewards show us that DPO is better over all, with respect to the objectives we set.

4 Best Arm Identification in Multi-armed Bandit (25pts)

In many experimental settings we are interested in quickly identifying the “best” of a set of potential interventions, such as finding the best of a set of experimental drugs at treating cancer, or the website design that maximizes user subscriptions. Here we may be interested in efficient pure exploration, seeking to quickly identify the best arm for future use.

In this problem, we bound how many samples may be needed to find the best or near-optimal intervention. We frame this as a multi-armed bandit with rewards bounded in $[0, 1]$. Recall a bandit problem can be considered as a finite-horizon MDP with just one state ($|\mathcal{S}| = 1$) and horizon 1: each episode consists of taking a single action and observing a reward. In the bandit setting – unlike in standard RL – the action (or “arm”) taken does not affect the distribution of future states. We assume a simple multi-armed bandit, meaning that $1 < |\mathcal{A}| < \infty$. Since there is only one state, a policy is simply a distribution over actions. There are exactly $|\mathcal{A}|$ different deterministic policies. Your goal is to design a simple algorithm to identify a near-optimal arm with high probability.

We recall Hoeffding's inequality: if $X_1, \dots, X_n$ are i.i.d. random variables satisfying $0 \leq X_i \leq 1$ with probability 1 for all i , $\bar{X} = \mathbb{E}[X_1] = \dots = \mathbb{E}[X_n]$ is the expected value of the random variables, and $\hat{X} = \frac{1}{n} \sum_{i=1}^n X_i$ is the sample mean, then for any $\delta > 0$ we have

$$\Pr \left(|\hat{X} - \bar{X}| > \sqrt{\frac{\log(2/\delta)}{2n}} \right) < \delta.$$

Assuming that the rewards are bounded in $[0, 1]$, we propose this simple strategy: pull each arm n_e times, and return the action with the highest average payout $\hat{r}_a$. The purpose of this exercise is to study the number of samples required to output an arm that is at least ϵ -optimal with high probability. Intuitively, as n_e increases the empirical average of the payout $\hat{r}_a$ converges to its expected value $\bar{r}_a$ for every action a , and so choosing the arm with the highest empirical payout $\hat{r}_a$ corresponds to approximately choosing the arm with the highest expected payout $\bar{r}_a$.

- a. (10 pts) We start by bounding the probability of the “bad event” in which the empirical mean of some arm differs significantly from its expected return. Starting from Hoeffding's inequality with n_e samples allocated

to every action, show that:

$$\Pr \left(\exists a \in \mathcal{A} \quad \text{s.t.} \quad |\hat{r}_a - \bar{r}_a| > \sqrt{\frac{\log(2/\delta)}{2n_e}} \right) < |\mathcal{A}|\delta.$$

Note that, depending on your derivation, you may come up with a tighter upper bound than $|\mathcal{A}|\delta$. This is also acceptable (as long as you argue that your bound is tighter), but showing the inequality above is sufficient.

- b. (15 pts) After pulling each arm (action) n_e times our algorithm returns the arm with the highest empirical mean:

$$a^\dagger = \arg \max_a \hat{r}_a$$

Notice that $a^\dagger$ is a random variable. Let $a^* = \arg \max_a \bar{r}_a$ be the true optimal arm. Suppose that we want our algorithm to return at least an ϵ -optimal arm with probability at least $1 - \delta'$, as follows:

$$\Pr \left(\bar{r}_{a^\dagger} \geq \bar{r}_{a^*} - \epsilon \right) \geq 1 - \delta' \quad (1)$$

How accurately do we need to estimate each arm in order to pick an arm that is ϵ -optimal? Then derive how many total samples we need total (across all arms) to return an ϵ -optimal arm with prob at least $1 - \delta'$ (that satisfies Equation 1). Express your result as a function of the number of actions, the required precision ϵ and the failure probability δ' .

- c. (0 pts) (Optional challenge, will not be graded) The above derivation only assumed the outcomes were bounded between 0 and 1. In practice people often assume outcomes are drawn from a parametric distribution, and under mild assumptions, one can use the central limit theorem to assume the average outcomes for an arm will follow a normal distribution. Repeat the above analysis under this assumption, for a multi-armed bandit with two arms. Is the resulting number of samples significantly smaller under these assumptions? In real settings it is often very expensive to run experiments. Do you think the method and bound derived in (a)-(b) would be preferable to making a normal assumption and why or why not?
- a. There seem to be several approaches that could show at least the required inequality, and a few of them involve advanced theorems. So I chose the most intuitive and simplest approach for myself:

- using independence and
- the $\Pr(\text{at least one of event } i \text{ happen}) = 1 - \Pr(\text{none of the event } i \text{ happens})$ idea.

We observe that the samples for each arm are drawn independently of the samples for any other arm. This means the empirical means $\hat{r}_a$ are independent random variables across different arms a . So the events $E_a = \{|\hat{r}_a - \bar{r}_a| > t\}$ are independent events for different arms $a \in \mathcal{A}$.

Let F_a be the complement event of E_a : $F_a = \{|\hat{r}_a - \bar{r}_a| \leq t\}$.

The probability of F_a is $\Pr(F_a) = 1 - \Pr(E_a)$. Since $\Pr(E_a) < \delta$, we have $\Pr(F_a) > 1 - \delta$.

The event $\mathcal{E}$ (at least one bad event E_a occurs) is the complement of the event where *all* good events F_a occur simultaneously.

Define new event $\mathcal{F} = \bigcap_{a \in \mathcal{A}} F_a$. Then $\mathcal{E} = \mathcal{F}^c$ (complementary), so

$$\Pr(\mathcal{E}) = 1 - \Pr(\mathcal{F}) = 1 - \Pr\left(\bigcap_{a \in \mathcal{A}} F_a\right).$$

Using the fact that $\Pr(F_a) > 1 - \delta$ for each arm:

$$\Pr\left(\bigcap_{a \in \mathcal{A}} F_a\right) > \prod_{a \in \mathcal{A}} (1 - \delta) = (1 - \delta)^{|\mathcal{A}|}$$

, the probability of the bad event $\mathcal{E}$ is:

$$\Pr(\mathcal{E}) = 1 - \Pr\left(\bigcap_{a \in \mathcal{A}} F_a\right) < 1 - (1 - \delta)^{|\mathcal{A}|}$$

This provides a tighter upper bound:

$$\Pr\left(\exists a \in \mathcal{A} \quad \text{s.t.} \quad |\hat{r}_a - \bar{r}_a| > \sqrt{\frac{\log(2/\delta)}{2n_e}}\right) < 1 - (1 - \delta)^{|\mathcal{A}|}$$

Why is $1 - (1 - \delta)^{|\mathcal{A}|}$ tighter than $|\mathcal{A}|\delta$

Using Bernoulli's inequality, $(1 - \delta)^{|\mathcal{A}|} \geq 1 - |\mathcal{A}|\delta$ for $\delta \in [0, 1]$ and integer $|\mathcal{A}| \geq 1$.

$$\Rightarrow -(1 - \delta)^{|\mathcal{A}|} \leq |\mathcal{A}|\delta - 1$$

$$\Rightarrow 1 - (1 - \delta)^{|\mathcal{A}|} \leq |\mathcal{A}|\delta.$$

The inequality is strict for $|\mathcal{A}| > 1$ and $\delta \in (0, 1)$.

Thus, the bound $1 - (1 - \delta)^{|\mathcal{A}|}$ derived using the independence of the arm samples is tighter than the bound $|\mathcal{A}|\delta$, unless $|\mathcal{A}| = 1$ or $\delta = 0, 1$.

b. According to the given algorithm, we choose

$$a^l = \arg \max_a \hat{r}_a, \quad a^* = \arg \max_a \bar{r}_a.$$

Now we want

$$\Pr(\bar{r}_{a^l} \geq \bar{r}_{a^*} - \epsilon) \geq 1 - \delta'.$$

A sufficient condition is that all empirical (sample) mean errors are bounded by $\epsilon/2$:

$$\forall a, \quad |\hat{r}_a - \bar{r}_a| \leq \frac{\epsilon}{2},$$

because then we will have:

$$|\hat{r}_{a^\dagger} - \bar{r}_{a^\dagger}| \leq \frac{\epsilon}{2} \iff -\frac{\epsilon}{2} \leq \hat{r}_{a^\dagger} - \bar{r}_{a^\dagger} \leq \frac{\epsilon}{2} \implies \bar{r}_{a^\dagger} \geq \hat{r}_{a^\dagger} - \frac{\epsilon}{2} \quad (1)$$

$$|\hat{r}_{a^*} - \bar{r}_{a^*}| \leq \frac{\epsilon}{2} \iff -\frac{\epsilon}{2} \leq \hat{r}_{a^*} - \bar{r}_{a^*} \leq \frac{\epsilon}{2} \implies \bar{r}_{a^*} \geq \hat{r}_{a^*} - \frac{\epsilon}{2} \quad (2)$$

$$(1) \wedge (2) \implies \bar{r}_{a^\dagger} \geq \hat{r}_{a^*} - \epsilon$$

Now applying the result from (a), we get

$$\Pr \left(\exists a : |\hat{r}_a - \bar{r}_a| > \frac{\epsilon}{2} \right) \leq |A|\delta.$$

On the RHS we can let $|A|\delta = \delta'$:

$$\Pr \left(\exists a : |\hat{r}_a - \bar{r}_a| > \frac{\epsilon}{2} \right) \leq \delta'.$$

Then we just need another sufficient requirement

$$\sqrt{\frac{\log(2(\delta'/|A|))}{2n_e}} \leq \frac{\epsilon}{2},$$

which gives us

$$n_e \geq \frac{2}{\epsilon^2} \log \left(\frac{2|A|}{\delta'} \right),$$

for each arm.

So total samples we need (across all arms) is at least $|A|n_e$, which is

$$\frac{2|A|}{\epsilon^2} \log \left(\frac{2|A|}{\delta'} \right).$$

- c. The central limit theorem (probability theory) states that, under appropriate conditions, the distribution of a normalized version of the sample mean converges to a standard normal distribution. Using this, we assume the 2 arms a_1 and a_2 both approximately follow a normal distribution.

That is to say

$$\hat{r}_{a_1} \sim \mathcal{N}(\bar{r}_{a_1}, \sigma_1^2/n_e), \quad \hat{r}_{a_2} \sim \mathcal{N}(\bar{r}_{a_2}, \sigma_2^2/n_e),$$

, where $\bar{r}_{a_i}$ are the true means, is a good approximation.

Then the difference of them follows

$$\hat{r}_{a_1} - \hat{r}_{a_2} \sim \mathcal{N}(\bar{r}_{a_1} - \bar{r}_{a_2}, \frac{\sigma_1^2 + \sigma_2^2}{n_e}).$$

For convenience, denote $\mu := \bar{r}_{a_1} - \bar{r}_{a_2}$ and $\sigma^2 := \sigma_1^2 + \sigma_2^2$.

Following that, using the z-score and confidence level formulation, we know that generally for confidence interval with probability $1 - \alpha$, there is

$$\Pr(|\hat{r}_{a_1} - \hat{r}_{a_2}| \leq z_{\alpha/2} \frac{\sigma}{\sqrt{n_e}}) = 1 - \alpha,$$

$$\Pr(|\hat{r}_{a_1} - \hat{r}_{a_2}| > z_{\alpha/2} \frac{\sigma}{\sqrt{n_e}}) = \alpha.$$

Compare it with a relaxed (sufficient) version of the inequality we want, similar with what was derived in sub problem (b):

$$\Pr(|\hat{r}_{a_1} - \hat{r}_{a_2}| > \frac{\epsilon}{2}) \leq \delta',$$

we can require that

$$\frac{\epsilon}{2} \geq z_{\delta'/2} \frac{\sigma}{\sqrt{n_e}},$$

and hence for n_e we obtain

$$\begin{aligned} \Rightarrow \frac{\epsilon^2}{4} &\geq z_{\delta'/2}^2 \frac{\sigma^2}{n_e} \\ \Rightarrow \frac{\epsilon^2}{4z_{\delta'/2}^2 \sigma^2} &\geq \frac{1}{n_e} \\ \Rightarrow n_e &\geq \frac{4z_{\delta'/2}^2 \sigma^2}{\epsilon^2} = \frac{4z_{\delta'/2}^2 (\sigma_1^2 + \sigma_2^2)}{\epsilon^2}. \end{aligned}$$

Therefore the total sample needed is at least 2 times of that, namely

$$\frac{2 \cdot 4z_{\delta'/2}^2 (\sigma_1^2 + \sigma_2^2)}{\epsilon^2}.$$

If generalize it to a bandits problem with $|A|$ arms, it would be

$$\frac{|A| \cdot 4z_{\delta'/2}^2 (\sigma^2)}{\epsilon^2},$$

where we assume a union bound on the variances $\sum \sigma_i^2 \leq \sigma^2$ for all arms for easier comparison.

Compared with the Hoeffding Bound, CLT bound:

- Requires less total samples if the variances of arm rewards are small enough.
- Can be tighter than Hoeffding Bound.
- Is parametric and thus easier to use in optimizations, such as allocating more pulls to high-variance arms.
- Is sensitive to the variances of arms regarding sample complexity, and requires knowing them beforehand or having good approximations of them.
- Needs n_e large enough to fulfill assumptions for the CLT.

So both have pros and cons. We might want to choose them according to the scenarios at hand.

Use CLT Bounds if:

- We have historical data or domain knowledge telling you rewards are roughly normal (e.g., physiological measurements, production yields).
- Sample budgets are large enough that the normal approximation is reliable.
- We can afford a mild risk of slight deviation from the nominal δ' in exchange for fewer samples.

Use Hoeffding Bounds if:

- Rewards are bounded but potentially very skewed or multimodal (e.g., click-through rates on ads).
- We need guaranteed finite-sample error control, without any distributional assumptions.

- Our sample budget is tight, and we can't afford the risk of an underestimated variance.

5 Challenges of RLHF (5 pts)

RLHF and DPO leverage humans providing pairwise comparisons.

- (2 pts) Who are human raters likely to be that are employed to provide pairwise preferences for generic LLM RLHF/DPO (in terms of socioeconomic profiles)? What ethical issues may this have, and what impact may this have on the resulting trained models?
 - (1 pt) Would RLHF be a good way to train a system to make better medical diagnoses? Why or why not?
 - (2 pts) There are a number of limitations of RLHF. Please describe one here (different than parts a and b). You may find it interesting to read ["Open Problems and Fundamental Limitations of Reinforcement Learning from Human Feedback"](#) and you are welcome to describe challenges listed there, or make up your own.
- a. Human raters for large-scale RLHF or DPO training on language models are often sourced from online platforms, like Mechanical Turk, Upwork, or specialized annotation companies ([Putting the human touch on LLMs](#)). Typically they are adults from diverse but often underpaid gig economy workers or crowdworkers, sometimes from developing countries. They may have varying levels of education (from high school to some college) but rarely experts in the model's domain. This means the raters might be economically vulnerable, working for relatively low wages under potentially stressful or monotonous conditions.

Based on this, I think there are at least these ethical issues:

- Exploitation. Low pay and precarious working conditions raise fairness and labor rights concerns.
- Bias introduction. Raters' cultural backgrounds, political views, and personal experiences influence their preferences, which can bias the model toward certain worldviews or values (e.g., Western-centric, politically biased).
- Lack of domain expertise. For specialized tasks (e.g., animation, legal), non-expert raters may produce misleading or suboptimal preferences.

First of all, suboptimal preferences will harm model quality. Secondly, the biases and limitations in raters' profiles can propagate into the model, resulting in systems that reflect specific cultural or ideological viewpoints, which would be bad for generalization in deployment.

- One can argue both ways using similar set of reasoning points. From my perspective it is easier to argue against it. Here are my reasons.
 - Medical diagnosis outcomes can be multi-faceted, uncertain, and context-dependent, making pairwise preferences hard to specify.
 - RLHF models risk learning over-fitted correlations, which is dangerous in high-stakes medical contexts.
 - If raters' preferences reflect local practices or biases, models may not generalize well or could amplify disparities in care.
- After reading the paper, I would like to address the risk of reward hacking in RLHF.

Since the reward model is trained from imperfect human feedback, it may assign high scores to outputs that superficially look good according to the learned preferences but are actually undesirable (e.g., verbose, evasive, or even toxic responses). The policy can exploit these flaws, producing outputs that maximize reward yet fail to align with true human intent.

A recent example illustrating this broader failure mode is the July 2025 incident involving Grok, the chatbot developed by xAI. As [reported in CNN](#), Grok produced several antisemitic outputs in public interactions. While the root cause is not publicly confirmed to be RLHF-related, the incident reflects a core risk: models can learn to game reward signals or mirror undesirable data distributions if the training process fails to account for edge cases or wholistic perspectives. It illustrates how reward hacking, or more broadly misalignment, can manifest in real-world deployments.

This cautions us that, even if models appear well-aligned in RLHF training scenarios, without robust generalization and safety mechanisms, they can behave unethically in deployment.

References

- Bradley, Ralph Allan, and Milton E. Terry. 1952. "Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons." *Biometrika* 39 (3/4): 324. <https://doi.org/10.2307/2334029>.
- Christiano, Paul, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. "Deep Reinforcement Learning from Human Preferences." arXiv. <https://doi.org/10.48550/ARXIV.1706.03741>.
- Rafailov, Rafael, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. "Direct Preference Optimization: Your Language Model Is Secretly a Reward Model." arXiv. <https://doi.org/10.48550/ARXIV.2305.18290>.